SAFEROOM IOT BOX

CẨM NANG THÁI KIỆT

Vấn đáp - Q&A - Code walkthrough - Kịch bản demo

CB-03 NC-01 NC-03 NC-05

Điều khiển quạt • Anomaly engine • MongoDB Time Series
Email cảnh báo và throttle 15 phút

Tài liệu hợp nhất từ bốn hướng dẫn trong thư mục `docs`

Cách dùng cẩm nang

Tài liệu được sắp theo bốn nhịp học: hiểu kiến trúc và câu trả lời cốt lõi; luyện 80 câu hỏi; theo dấu code; cuối cùng là chạy demo và xử lý lỗi. Hãy hiểu luồng dữ liệu trước, nhớ contract và giới hạn sau, rồi mới học vị trí file.

Ôn nhanh trước vấn đáp	Đọc Phần I và cheat sheet cuối phần.
Luyện phản biện	Đọc Phần II - 80 câu hỏi và trả lời.
Chuẩn bị mở code	Đọc Phần III - Code walkthrough.
Chuẩn bị demo	Đọc Phần IV - Kịch bản demo.

Mục lục

Cách dùng cẩm nang 1

Cẩm nang vấn đáp phần Thái Kiệt 1

1. Phạm vi phụ trách . . . 2
2. Câu giới thiệu 30 giây . . . 3
3. Sơ đồ tổng thể phần Kiệt . . . 3
4. CB-03 - Điều khiển quạt từ Website . . . 4
5. NC-01 - Anomaly engine và Z-score . . . 6
6. NC-03 - MongoDB Time Series . . . 8
7. NC-05 - Email Notification . . . 9
8. Bản đồ file cần mở khi vấn đáp . . . 10
9. Bằng chứng hiện có . . . 11
10. Cheat sheet một phút trước khi vào phòng . . . 12

Bộ câu hỏi vấn đáp phần Thái Kiệt 13

A. Câu hỏi tổng quan . . . 14
B. CB-03 - Điều khiển quạt . . . 15
C. NC-01 - Z-score và anomaly . . . 17
D. NC-03 - MongoDB Time Series . . . 20
E. NC-05 - Email . . . 22
F. Câu hỏi phản biện và cách trả lời trung thực . . . 24
Câu hỏi bổ sung về MQ2 sau hiệu chỉnh . . . 25

Code walkthrough phần Thái Kiệt 26

1. Cách đọc project . . . 27
2. CB-03 - Điều khiển quạt . . . 28
3. NC-01 - Z-score và rule engine . . . 33
4. NC-03 - Time Series ingestion . . . 36
5. NC-05 - Nodemailer và throttle . . . 38
6. Test map . . . 40
6.1 MQ2 readiness contract cần biết khi giải thích telemetry . . . 41
7. Điều nên và không nên học thuộc . . . 42

Kịch bản demo phần Thái Kiệt 43

1. Chuẩn bị trước demo . . . 44
2. Demo nhanh không cần phần cứng . . . 45
3. Show từng phần bằng test có mục tiêu . . . 46
4. Build verification . . . 48

5. Demo thật CB-03 / TC-06 49
6. Demo thật NC-03 / TC-02 51
7. Demo thật NC-01 / TC-04 52
8. Demo thật NC-05 / TC-05 53
9. Bảng ghi kết quả vật lý 54
10. Cách xử lý khi demo lỗi 55
11. Bằng chứng dự phòng 56

Cẩm nang vấn đáp phần Thái Kiệt

Tài liệu này bám theo proposal của nhóm và phiên bản code hiện tại của SafeRoom IoT Box. Mục tiêu là giúp Thái Kiệt giải thích đúng phần mình phụ trách, chỉ được code liên quan, nói rõ giới hạn và đưa ra bằng chứng kiểm thử.

1. Phạm vi phụ trách

ID	Chức năng	Điểm trong proposal	Phần cần nắm chắc
CB-03	Điều khiển quạt từ Website	4.0	React -> REST API -> MQTT QoS 1 -> ESP32 -> relay GPIO 16
NC-01	Phát hiện dữ liệu bất thường bằng Z-score	2.0	Rolling window N=30, Z-score, hard rules, dedupe, hành động tự động
NC-03	Lưu dữ liệu cảm biến theo thời gian	1.5	MongoDB Time Series, schema, ingestion, retention 30 ngày, historical query
NC-05	Cảnh báo tự động qua Email	1.0	Nodemailer, HIGH/CRITICAL, HTML template, throttle 15 phút

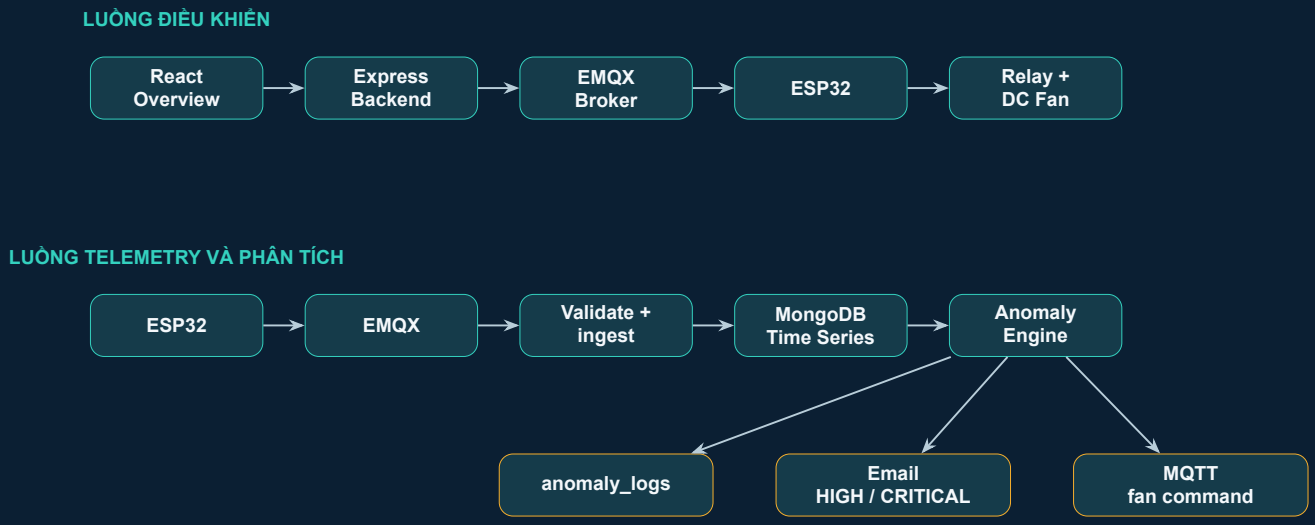
Phần của Kiệt đi xuyên nhiều tầng. Không nên trả lời “em chỉ làm backend”. Câu trả lời chính xác hơn:

> *Em phụ trách thiết kế và tích hợp bốn luồng CB-03, NC-01, NC-03 và NC-05. Trọng tâm code của em nằm ở backend, nhưng CB-03 còn có phần gọi API ở React và phần nhận lệnh MQTT trên ESP32. Em cũng viết test để chứng minh các tầng nối với nhau đúng contract.*

2. Câu giới thiệu 30 giây

> *Phần của em nhận telemetry từ ESP32 qua MQTT, kiểm tra và lưu vào MongoDB Time Series. Sau khi lưu, anomaly engine áp dụng hard rule cho gas và motion, còn nhiệt độ và độ ẩm có rolling Z-score N=30. Khi có anomaly HIGH hoặc CRITICAL, backend có thể bật quạt qua MQTT và gửi email, trong đó email được giới hạn một lần mỗi 15 phút theo từng cặp thiết bị và loại cảm biến. Với CB-03, người dùng bật quạt trên React, backend publish lệnh QoS 1, ESP32 nhận JSON và đổi GPIO 16.*

3. Sơ đồ tổng thể phần Kiệt



Ba ý cần nhấn mạnh:

1. MQTT tách backend khỏi ESP32, vì vậy web không điều khiển GPIO trực tiếp.
2. Telemetry được validate trước khi lưu và trước khi chạy anomaly engine.
3. Gas vẫn có cảnh báo cục bộ trên ESP32; backend không được xem là lớp an toàn duy nhất.

4. CB-03 - Điều khiển quạt từ Website

4.1 Luồng xử lý

1. Người dùng nhấn nút fan trên `Overview.jsx`.
2. Frontend đảo trạng thái hiện tại và gửi `POST/api/v1/control/actuator`.
3. JWT được gắn vào request bởi `apiFetch()`.
4. Backend kiểm tra `deviceId`, actuator và state.
5. `mqttService.publishControlCommand()` publish JSON lên `safebox/{deviceId}/control`, QoS 1.
6. `MqttClient` trên ESP32 parse payload rồi gọi callback `onControlMessage()` trong `main.cpp`.
7. Callback định tuyến actuator `fan` sang module `Relay`; actuator lạ bị bỏ qua.
8. `Relay::on()` hoặc `Relay::off()` ghi GPIO 16 theo cấu hình `RELAY_ACTIVE_LOW`.
9. Backend tạm ghi trạng thái yêu cầu để UI phản hồi nhanh; telemetry kế tiếp từ ESP32 xác nhận lại trạng thái thật.

4.2 Contract quan trọng

Request từ frontend:

```
{
  "deviceId": "safebox-01",
  "actuator": "fan",
  "state": true
}
```

MQTT command gửi xuống ESP32:

```
{
  "actuator": "fan",
  "state": "on"
}
```

Topic:

```
safebox/safebox-01/control
```

4.3 Vì sao REST dùng boolean nhưng MQTT dùng on/off?

- Boolean thuận tiện cho toggle React và validation API.
- Chuỗi `on`/`off` dễ đọc trên MQTT client và Serial Monitor.
- Backend là nơi chuyển đổi contract, nên ESP32 không phải hiểu nhiều định dạng.

4.4 Vì sao dùng QoS 1?

QoS 1 bảo đảm lệnh được giao ít nhất một lần giữa client và broker. Lệnh bật/tắt là idempotent: nhận `fan=on` hai lần vẫn chỉ cho fan ở trạng thái ON. Không được nói QoS 1 là “exactly once”; đó là QoS 2.

4.5 Requested state và confirmed state

Sau khi broker nhận publish thành công, backend cập nhật requested state để API status phản hồi ngay. Tuy nhiên đây chưa phải bằng chứng relay đã đổi trạng thái. Telemetry kế tiếp chứa `actuatorStates.fanOn` sẽ reconcile lại state theo báo cáo của ESP32.

Đây là câu trả lời tốt khi bị hỏi “publish thành công có chắc quạt quay chưa?”:

> *Chưa chắc. Publish callback chỉ chứng minh broker nhận lệnh. Xác nhận thiết bị cần telemetry phản hồi hoặc acknowledgment riêng. Bản hiện tại dùng telemetry kế tiếp để reconcile; nếu cần xác nhận mạnh hơn có thể thêm command ID và ACK topic.*

4.6 Fail-safe của firmware

- GPIO 16 được cấu hình OUTPUT khi khởi động.
- `Relay::begin()` gọi `off()` nên fan mặc định OFF; với HL-52S active-low, mức boot là HIGH.
- JSON sai bị bỏ qua; actuator lạ không có handler. Backend chỉ publish state hợp lệ `on`/`off`.
- Không có local auto-trigger cho fan ngoài yêu cầu backend, đúng phạm vi dự án.

5. NC-01 - Anomaly engine và Z-score

5.1 Công thức

```
mean = tổng 30 giá trị / 30
variance = tổng (x_i - mean)^2 / 30
standard deviation = sqrt(variance)
Z = (giá trị mới - mean) / standard deviation
```

Một điểm được xem là anomaly thống kê khi đồng thời:

```
|Z| > zScoreLimit
và
|giá trị mới - mean| > minimum physical delta
```

Giá trị mặc định:

Thông số	Giá trị
Window	30 mẫu trước đó
Z-score limit	3.0
Minimum delta nhiệt độ	1.0°C
Minimum delta độ ẩm	2.0%

5.2 Vì sao dùng 30 mẫu trước, không cho mẫu mới vào trước?

Mẫu mới phải được so với baseline trước đó. Nếu đưa spike vào window trước khi tính mean và độ lệch chuẩn, chính spike sẽ kéo baseline về phía nó và làm giảm Z-score. Code tính thống kê trước, sau đó mới `push` mẫu mới và bỏ mẫu cũ nhất.

5.3 Cold start và zero variance

- Ít hơn 30 mẫu: chưa đủ baseline, trả `ready=false` và dựa vào hard threshold.
- Độ lệch chuẩn bằng 0: không chia cho 0; trả Z=0 và không tạo anomaly thống kê.
- Hard threshold vẫn luôn hoạt động, kể cả khi Z-score chưa sẵn sàng.

5.4 Vì sao gas và motion không dùng Z-score?

- Nhiệt độ và độ ẩm là biến liên tục, phù hợp với phân tích độ lệch quanh baseline.
- Motion là boolean, không có ý nghĩa Gaussian.
- Gas là biến an toàn có ngưỡng cứng 600 PPM; chờ đủ 30 mẫu hoặc chờ Z-score có thể gây nguy hiểm.

5.5 Rule matrix thực tế

Sensor	Điều kiện	Trigger	Severity	Hành động
Gas	`gasLevel > gasMax`	Hard threshold	CRITICAL	Log + email
Temperature	Ngoài min/max	Hard threshold	HIGH	Log + email; bật fan nếu vượt max
Temperature	Auto mode và Z bất thường	Z-score	HIGH	Log + email; bật fan nếu spike hướng lên
Motion	Motion=true và system armed	Hard threshold	HIGH	Log + email + alert LED
Humidity	Ngoài min/max	Hard threshold	MEDIUM	Log
Humidity	Auto mode và Z bất thường	Z-score	MEDIUM	Log

5.6 Dedupe khác email throttle thế nào?

- Anomaly dedupe mặc định 60 giây, key gồm `deviceId:sensorType:triggerType`. Mục đích là tránh ghi cùng sự kiện quá dày.
- Email throttle là 15 phút, key gồm `deviceId:sensorType`. Mục đích là tránh spam hộp thư.
- Hai cơ chế giải quyết hai vấn đề khác nhau và không nên gộp.

5.7 Thứ tự xử lý anomaly có cố định không?

Không. Các candidate được xử lý bằng `Promise.all`, nên email hoặc MQTT của từng candidate có thể hoàn tất khác thứ tự. Điều cần bảo đảm là đủ tập anomaly và đúng nội dung, không dựa vào thứ tự hoàn thành.

6. NC-03 - MongoDB Time Series

6.1 Tại sao dùng Time Series collection?

Telemetry là dữ liệu liên tục, mỗi document có timestamp và deviceId. Time Series tối ưu cách MongoDB lưu và truy vấn dữ liệu theo thời gian, đồng thời hỗ trợ retention tự động.

6.2 Cấu hình authoritative

```
collection: sensor_readings
timeField: timestamp
metaField: deviceId
granularity: seconds
expireAfterSeconds: 2,592,000 (30 ngày)
```

6.3 Luồng ghi dữ liệu

1. `mqttService` nhận topic telemetry.
2. `validateTelemetryPayload()` kiểm tra device ID, timestamp, metric và actuator state.
3. `SensorReading.create()` ghi document vào `sensor_readings`.
4. Document vừa ghi được chuyển cho anomaly engine.

Việc lưu trước rồi phân tích giúp giữ lại dữ liệu gốc ngay cả khi một hành động anomaly như email hoặc MQTT gặp lỗi.

6.4 Vì sao schema đặt `autoCreate=false`?

Mongoose không tự tạo collection thường. `timeSeriesService` chủ động tạo collection với đúng `timeseries` và TTL option trước khi model ghi dữ liệu. Nếu collection đã tồn tại nhưng sai cấu hình, backend fail fast và yêu cầu migration thay vì âm thầm chạy sai.

6.5 Historical query lấy dữ liệu thế nào?

Backend lọc theo device và khoảng thời gian, sort giảm dần để lấy đúng N mẫu mới nhất, áp dụng limit, sau đó reverse trong bộ nhớ để frontend nhận chuỗi theo thời gian tăng dần.

```
MongoDB: newest -> oldest, limit N
Response: oldest -> newest trong N mẫu vừa chọn
```

Nếu sort tăng dần trước limit, API sẽ trả N mẫu cũ nhất trong khoảng, không phải N mẫu mới nhất.

6.6 TTL có xóa đúng ngay giây thứ 2,592,000 không?

Không nên khẳng định chính xác tới từng giây. MongoDB TTL monitor chạy nền, nên document được xóa sau khi hết hạn với một độ trễ nhỏ. Ý nghĩa là retention tự động, không phải timer realtime.

7. NC-05 - Email Notification

7.1 Luồng gửi email

1. Anomaly candidate được tạo.
2. `executeActions()` chỉ gọi email cho severity HIGH hoặc CRITICAL.
3. `sendAlertEmail()` đọc cấu hình SMTP sender từ environment.
4. Recipient resolver lấy `User.email` của các tài khoản có `settings.emailEnabled=true`.
5. Hàm tạo key `deviceId:sensorType` và kiểm tra lần gửi gần nhất.
6. Nếu chưa bị throttle, Nodemailer gửi cả text và HTML email.
7. Chỉ sau khi `sendMail()` thành công mới lưu thời điểm gửi.

7.2 Vì sao chỉ HIGH và CRITICAL?

MEDIUM như humidity bất thường vẫn được ghi log và hiển thị trên dashboard nhưng không gửi email để tránh cảnh báo quá mức. HIGH và CRITICAL cần người dùng phản ứng sớm hơn.

7.3 Vì sao throttle theo cặp device và sensor?

Gas trên `safebox-01` không được chặn email nhiệt độ trên cùng thiết bị, và cũng không được chặn gas trên `safebox-02`. Vì vậy key phải chứa cả device và sensor.

Ví dụ:

```
safebox-01:gasLevel       -> một window riêng
safebox-01:temperature    -> một window riêng
safebox-02:gasLevel       -> một window riêng
```

7.4 Vì sao lưu throttle sau khi gửi thành công?

Nếu SMTP lỗi mà vẫn ghi thời điểm throttle, các lần retry tiếp theo sẽ bị chặn dù người dùng chưa nhận email nào. Code chỉ cập nhật map sau `sendMail()` thành công.

7.5 Giới hạn cần nói thật

- Throttle map nằm trong RAM, nên reset khi backend restart.
- Recipient lấy từ email đăng ký của mọi user đang bật email trong Settings; nếu tất cả user tắt thì SMTP không được gọi.
- Hệ thống chưa có quan hệ device-owner, nên mọi user đang bật đều nhận anomaly của thiết bị. Bản production cần ánh xạ user với device.
- `ALERT_RECIPIENT_EMAIL` chỉ là fallback khi database chưa có tài khoản nào.
- Nếu triển khai nhiều instance, nên chuyển throttle sang Redis/MongoDB.

8. Bản đồ file cần mở khi vấn đáp

Phần	File chính	Vai trò
CB-03 frontend	`frontend/src/pages/Overview.jsx`	Toggle, POST API, optimistic state
CB-03 auth helper	`frontend/src/api.js`	Gắn JWT vào request
CB-03 API	`backend/routes/control.js`	Validate request và gọi MQTT service
CB-03 state	`backend/services/actuatorStateService.js`	Requested state và telemetry reconciliation
CB-03 MQTT	`backend/services/mqttService.js`	Publish QoS 1 và nhận telemetry/status
CB-03 firmware	`firmware/src/main.cpp`	Đăng ký callback và định tuyến actuator
CB-03 MQTT parser	`firmware/src/mqtt/mqtt_client.cpp`	Subscribe control topic và parse JSON
CB-03 relay	`firmware/src/actuators/relay.cpp`	Ghi GPIO 16 theo polarity cấu hình
NC-01	`backend/services/anomalyEngine.js`	Z-score, rules, dedupe, actions
NC-01 schema	`backend/models/Schemas.js`	Cấu trúc `anomaly_logs`
NC-03	`backend/services/timeSeriesService.js`	Tạo và verify Time Series collection
NC-03 ingestion	`backend/services/telemetryService.js`	Validate, create, gọi anomaly engine
NC-03 API	`backend/routes/telemetry.js`	Latest và historical query
NC-05	`backend/services/emailService.js`	SMTP, template, severity, throttle
Khởi động	`backend/server.js`	MongoDB, Time Series, email, MQTT, mock

Chi tiết code có trong `THAI_KIET_CODE_WALKTHROUGH.md`.

9. Bằng chứng hiện có

Chạy tại thư mục `backend`:

```
npm run test:kiet
```

Bộ test chứng minh:

- Mapping và reconcile actuator state.
- JWT chặn request không xác thực trong CB-03 simulation.
- N=30, per-device buffer, zero variance và minimum delta.
- Gas/motion không đi qua Z-score.
- Hard-rule matrix và hành động alert LED.
- Temperature spike tạo Z-score anomaly, bật fan, ghi log và gửi email.
- Email throttle cùng key và độc lập giữa các key.
- Time Series options và retention 30 ngày.
- Historical API chọn N mẫu mới nhất và trả theo thứ tự thời gian.

Điều chưa được test phần mềm thay thế hoàn toàn:

- Relay vật lý có click và fan quay dưới 500 ms hay không.
- SMTP thật có giao email trong dưới 5 giây hay không.
- EMQX và Atlas thật có ổn định trong phiên demo hay không.

Các mục này phải được đo bằng TC-02, TC-04, TC-05 và TC-06 với hạ tầng thật.

10. Cheat sheet một phút trước khi vào phòng

```
CB-03:
React POST boolean -> backend validate -> MQTT on/off QoS 1
-> ESP32 MqttClient -> onControlMessage -> Relay -> GPIO 16
Publish success != physical ACK; telemetry kế tiếp reconcile state.

NC-01:
Z-score chỉ temp/humidity, previous N=30, |Z|>3 và min delta.
Gas >600 và motion dùng hard rule.
Cold start dùng hard bounds; sigma=0 thì Z=0.
Dedupe anomaly 60s khác email throttle 15m.

NC-03:
sensor_readings Time Series; timestamp=timeField; deviceId=metaField.
Granularity seconds; TTL 30 ngày.
Validate -> create -> anomaly engine.
Sort DESC + limit + reverse để lấy N mẫu mới nhất theo thứ tự tăng dần.

NC-05:
Nodemailer chỉ HIGH/CRITICAL.
Throttle key=deviceId:sensorType, 15 phút.
Chỉ ghi lastSent sau khi SMTP gửi thành công.

MQ2 telemetry:
48h burn-in -> stable clean-air Ro -> NVS -> 5m warm-up on later boots.
Only ready has numeric gasLevel; preparing sends null + sensorStatus.
100ms high-priority local task; raw fallback stays local, no fake email.
```

Bộ câu hỏi vấn đáp phần Thái Kiệt

Câu trả lời mẫu được viết theo cách nói trực tiếp. Không cần học thuộc từng chữ; cần hiểu luận điểm và chỉ được code minh họa.

A. Câu hỏi tổng quan

1. Em phụ trách phần nào?

Em phụ trách CB-03 điều khiển quạt từ website, NC-01 anomaly detection bằng Z-score và hard rules, NC-03 lưu telemetry bằng MongoDB Time Series, và NC-05 gửi email cảnh báo có throttle.

2. Bốn phần này liên hệ với nhau thế nào?

NC-03 nhận và lưu telemetry. NC-01 phân tích chính telemetry đó. NC-01 có thể gọi lại luồng CB-03 để bật fan khi nhiệt độ cao và gọi NC-05 để gửi email khi severity HIGH hoặc CRITICAL.

3. Thành phần nào là trung tâm điều phối?

Express backend. Nó nhận REST từ frontend, giao tiếp MQTT với ESP32, ghi MongoDB, chạy anomaly engine và gọi Nodemailer.

4. Vì sao không cho frontend kết nối MQTT trực tiếp?

Backend cần giữ MQTT credential, validate quyền người dùng, chuẩn hóa lệnh và tập trung business rule. Nếu frontend kết nối broker trực tiếp thì credential bị lộ và khó kiểm soát quyền publish.

5. Điểm an toàn quan trọng nhất của kiến trúc là gì?

Cảnh báo gas nguy hiểm vẫn phải chạy cục bộ trên ESP32 dưới 200 ms, không phụ thuộc backend. Backend bổ sung lưu log và email chứ không thay thế edge safety.

6. Em chứng minh code chạy bằng gì?

Em dùng test tự động cho logic và integration simulation, cộng với firmware/frontend build. Riêng latency relay, SMTP thật, EMQX và Atlas phải đo bằng test end-to-end với hạ tầng thật.

7. Nếu được làm lại, em ưu tiên cải tiến gì?

Em sẽ thêm command ID và ACK topic cho actuator, chuyển dedupe/throttle state sang Redis hoặc MongoDB để không mất khi restart, và gắn device ownership với user cho mô hình nhiều thiết bị.

B. CB-03 - Điều khiển quạt

8. Luồng bật quạt bắt đầu từ đâu?

Từ `handleActuatorToggle('fan', actuators.fanOn)` trong `Overview.jsx`. Frontend gửi POST tới `/api/v1/control/actuator` với deviceId, actuator và boolean state.

9. Request có được xác thực không?

Có. `apiFetch()` gắn JWT vào Authorization header. Backend mount `authenticateToken` trước control router. Không có token trả 401, token sai hoặc hết hạn trả 403.

10. Backend validate những gì?

Route yêu cầu actuator và state, actuator phải nằm trong allowlist, state phải là boolean hoặc chuỗi on/off. MQTT service kiểm tra thêm deviceId, actuator và boolean trước khi publish.

11. Topic điều khiển là gì?

`safebox/{deviceId}/control`, ví dụ `safebox/safebox-01/control`.

12. Payload MQTT là gì?

`{"actuator":"fan","state":"on"}` hoặc state `off`.

13. Vì sao dùng QoS 1?

Lệnh điều khiển quan trọng hơn telemetry thông thường, nên cần at-least-once delivery. Bật/tắt là idempotent nên duplicate không gây toggle ngược.

14. QoS 1 có bảo đảm quạt đã quay không?

Không. Nó chỉ bảo đảm giao message ở tầng MQTT đến broker/client theo cơ chế QoS. Muốn biết phần cứng đã thực thi phải dùng telemetry phản hồi hoặc ACK riêng từ ESP32.

15. Vì sao control message không retained?

Nếu retained, ESP32 kết nối lại có thể nhận lệnh cũ và bật quạt ngoài ý muốn. Control command là event hiện tại nên dùng `retain=false`.

16. Nếu MQTT offline thì API trả gì?

`publishControlCommand()` ném lỗi có code `MQTT_OFFLINE`, route chuyển thành HTTP 503. Frontend hiển thị control feedback lỗi và không giả vờ thành công.

17. Trạng thái fan trên backend lấy từ đâu?

Ngay sau publish thành công, backend lưu requested state trong `actuatorStateService`. Khi telemetry ESP32 về, snapshot được synchronize lại bằng `actuatorStates` do thiết bị báo cáo.

18. Vì sao tách `actuatorStateService` khỏi mock generator?

State này được dùng cả simulation và MQTT thật. Đặt nó trong file mock gây hiểu nhầm ownership. Service riêng làm rõ đây là trạng thái runtime chung; mock generator chỉ sinh dữ liệu.

19. Firmware chống payload sai thế nào?

ArduinoJson parse theo buffer và length; JSON lỗi bị bỏ qua. `main.cpp` chỉ định tuyến `fan`, `buzzer`, `alertLed`, nên actuator lạ không tạo tác động. Backend đã validate và chỉ publish state hợp lệ; phía firmware xử lý chuỗi khác `on` theo hướng fail-off.

20. Fan ở trạng thái nào khi ESP32 boot?

OFF. `Relay::begin()` cấu hình pin OUTPUT rồi gọi `off()`. Với HL-52S mặc định active-low, OFF tương ứng GPIO 16 HIGH; polarity có thể đổi bằng `RELAY_ACTIVE_LOW`.

21. Nếu QoS 1 giao cùng lệnh hai lần thì sao?

Không sao vì command mang trạng thái đích, không phải lệnh toggle. Gọi `Relay::on()` hai lần vẫn cho fan ON.

22. Hệ thống có hỗ trợ nhiều device hoàn chỉnh chưa?

Topic, telemetry schema và historical query có deviceId. Tuy nhiên runtime actuator snapshot hiện là một object vì coursework dùng một device chính. Muốn multi-device hoàn chỉnh cần đổi thành `Map<deviceId, actuatorStates>`.

C. NC-01 - Z-score và anomaly

23. Công thức Z-score là gì?

`Z=(x-mean)/standardDeviation`. Code lấy mean và standard deviation từ 30 mẫu trước đó của cùng device và cùng sensor.

24. Vì sao chọn N=30?

Đây là kích thước rolling window theo thiết kế dự án: đủ tạo baseline ngắn hạn nhưng vẫn phản ứng với thay đổi môi trường. Với chu kỳ 2 giây, 30 mẫu đại diện khoảng một phút.

25. Tại sao dùng 30 mẫu trước thay vì gồm cả mẫu mới?

Mẫu mới là đối tượng cần kiểm tra. Nếu đưa nó vào baseline trước, spike làm mean và variance tăng, khiến Z-score bị giảm và khó phát hiện hơn.

26. Điều kiện anomaly thống kê là gì?

Hai điều kiện đồng thời: trị tuyệt đối Z lớn hơn threshold mặc định 3.0 và độ lệch vật lý lớn hơn minimum delta, 1°C cho nhiệt độ hoặc 2% cho độ ẩm.

27. Minimum delta để làm gì?

Khi standard deviation rất nhỏ, một dao động rất nhỏ cũng có thể tạo Z-score lớn. Minimum delta lọc các spike toán học nhưng không có ý nghĩa vật lý.

28. Z-score áp dụng cho sensor nào?

Chỉ temperature và humidity. `MIN_DELTA` cũng đóng vai trò allowlist; sensor khác trả `ready=false` ngay.

29. Vì sao gas không dùng Z-score?

Gas là thông số an toàn có ngưỡng cứng. Gas vượt 600 PPM phải cảnh báo ngay, không thể chờ 30 mẫu hoặc phụ thuộc phân phối thống kê.

30. Vì sao motion không dùng Z-score?

Motion là boolean, không phải biến liên tục có phân phối phù hợp để tính mean và standard deviation theo cách này.

31. Cold start xử lý thế nào?

Khi buffer dưới 30 mẫu, Z-score chưa ready. Hard bounds vẫn chạy để không bỏ qua nhiệt độ, độ ẩm hoặc gas nguy hiểm.

32. Standard deviation bằng 0 thì sao?

Phép chia không xác định nên code trả Z=0 và không tạo statistical anomaly. Hard thresholds vẫn là lớp fallback. Có thể dùng epsilon trong hệ thống nâng cao, nhưng bản này tuân theo rule zero-variance của dự án.

33. Code dùng population hay sample standard deviation?

Population standard deviation, variance chia N. Rolling window được xem là toàn bộ baseline runtime hiện tại, không phải mẫu nghiên cứu cần ước lượng tổng thể bằng N-1.

34. Buffer có trộn dữ liệu giữa hai thiết bị không?

Không. Key là `deviceId:sensorType`, ví dụ `room-a:temperature` khác `room-b:temperature` và khác `room-a:humidity`.

35. Automatic mode có tắt hard threshold không?

Không. Hard threshold luôn được ưu tiên như safety bound. Automatic mode chỉ bật thêm nhánh Z-score khi giá trị chưa vi phạm hard bound.

36. Nhiệt độ Z-score âm có bật fan không?

Không. Temperature anomaly vẫn được log và email nếu HIGH, nhưng `autoFan` chỉ true khi giá trị mới lớn hơn mean, tức bất thường theo hướng nóng lên.

37. Gas vượt ngưỡng tạo severity gì?

CRITICAL, trigger type `HARD_THRESHOLD_BREACH`. Edge alert từ ESP32 dùng trigger type `EDGE_SAFETY_TRIGGER` nhưng cũng CRITICAL.

38. Motion tạo anomaly khi nào?

Chỉ khi `motionDetected=true` và system đang armed. Nó tạo HIGH anomaly, gửi email và publish lệnh bật alert LED.

39. Humidity anomaly gửi email không?

Không trong matrix hiện tại vì severity MEDIUM. Nó vẫn được lưu vào `anomaly_logs` và hiện trên dashboard.

40. Dedupe key gồm gì?

`deviceId:sensorType:triggerType`. Như vậy hard threshold và edge trigger của cùng gas vẫn là hai loại sự kiện khác nhau.

41. Vì sao reserve dedupe key trước action async?

Nếu hai telemetry message vào gần như đồng thời, cả hai có thể cùng vượt kiểm tra trước khi lần đầu ghi DB. Reserve trước giúp chặn race duplicate.

42. Nếu action hoặc ghi DB lỗi thì dedupe key thế nào?

Catch sẽ xóa reservation nếu nó vẫn thuộc lần xử lý hiện tại, cho phép message sau retry thay vì bị chặn oan.

43. Các anomaly có luôn được tạo đúng thứ tự gas, temp, motion, humidity không?

Không nên phụ thuộc thứ tự hoàn tất vì dùng `Promise.all`. Business correctness nằm ở tập kết quả và nội dung từng anomaly.

44. Nếu bật fan thất bại thì anomaly có bị mất không?

Không. Lỗi action được bắt và ghi vào `actionsTaken`; pipeline tiếp tục tạo anomaly log. Đây là graceful degradation.

45. Điểm hạn chế của rolling buffer hiện tại là gì?

Buffer nằm trong RAM nên mất khi backend restart và mỗi instance có buffer riêng. Production nhiều instance nên lưu state vào Redis hoặc dùng truy vấn window từ time-series database.

D. NC-03 - MongoDB Time Series

46. Collection telemetry tên gì?

`sensor_readings`.

47. `timeField` và `metaField` là gì?

`timestamp` là timeField; `deviceId` là metaField. MongoDB dùng timestamp để tổ chức theo thời gian và deviceId để nhóm metadata series.

48. Granularity là gì?

`seconds`, phù hợp dữ liệu ESP32 gửi mỗi khoảng 2 giây.

49. Retention là bao lâu?

2,592,000 giây, tương đương 30 ngày.

50. TTL có xóa đúng ngay khi đủ 30 ngày không?

Không chính xác từng giây vì TTL monitor chạy nền. Document sẽ được xóa tự động sau khi hết hạn với độ trễ vận hành nhỏ.

51. Vì sao Mongoose schema dùng `autoCreate=false`?

Để Mongoose không tạo nhầm collection thường. Backend chủ động tạo Time Series collection với đúng options trước khi ghi.

52. Startup kiểm tra collection ra sao?

`ensureSensorReadingsTimeSeries()` đọc metadata. Nếu chưa có thì tạo; nếu có thì so timeField, metaField, granularity và TTL. Sai bất kỳ option nào thì fail fast.

53. Vì sao fail fast thay vì tự sửa collection?

Đổi một collection thường thành Time Series không phải update option đơn giản và có nguy cơ mất dữ liệu. Fail fast buộc migration có chủ đích.

54. Telemetry được validate trước hay sau khi lưu?

Trước. `validateTelemetryPayload()` chuẩn hóa dữ liệu, kiểm tra type/range và đối chiếu deviceId trong payload với topic.

55. Vì sao phải đối chiếu deviceId payload với topic?

Để client không publish vào topic của device A nhưng khai payload device B, gây ghi sai series hoặc giả mạo dữ liệu.

56. Tại sao historical query sort DESC rồi reverse?

DESC + limit lấy đúng N mẫu mới nhất. Reverse sau đó trả chúng theo thời gian tăng dần để chart vẽ đúng. ASC + limit sẽ lấy N mẫu cũ nhất.

57. Historical API giới hạn bao nhiêu record?

`limit` phải là integer từ 1 đến 1000. Giới hạn ngăn response quá lớn và query thiếu kiểm soát.

58. Nếu chỉ hỏi một sensor thì API làm gì?

Route dùng allowlist map sensor sang path schema rồi `.select()` timestamp, deviceId và field sensor đó. Không lấy field tùy ý từ query string.

59. Dữ liệu được lưu trước hay phân tích anomaly trước?

Lưu trước, sau đó chạy anomaly engine trên reading đã tạo. Như vậy dữ liệu gốc vẫn có trong DB nếu action phía sau gặp lỗi.

E. NC-05 - Email

60. Thư viện gửi email là gì?

Nodemailer với SMTP transport.

61. Severity nào gửi email?

HIGH và CRITICAL. LOW/MEDIUM trả status `ignored` trước khi liên hệ SMTP.

62. Throttle window là bao lâu?

15 phút, tức `15 * 60 * 1000` millisecond.

63. Throttle key gồm gì?

`deviceId:sensorType`. Nó bảo đảm các cảm biến và thiết bị khác nhau không chặn email của nhau.

64. Vì sao key không chứa triggerType?

Mục tiêu là hạn chế email theo vấn đề cảm biến. Nếu cùng gas vừa có hard threshold vừa có edge alert, người dùng vẫn không nên nhận hai chuỗi email riêng dồn dập.

65. Thời điểm throttle được ghi khi nào?

Sau khi `sendMail()` thành công. SMTP lỗi không được phép mở một window giả.

66. Nếu email bị disable thì anomaly có mất không?

Không. Email service trả status `disabled`; anomaly engine ghi `EMAIL_DISABLED` trong actionsTaken rồi vẫn lưu anomaly.

67. SMTP password nằm ở đâu?

Trong environment variable, không commit vào Git. `.env.example` chỉ chứa placeholder.

Địa chỉ gửi (`SMTP_USER`) vẫn do server quản lý, nhưng địa chỉ nhận lấy từ `User.email` đã đăng ký. Người dùng bật hoặc tắt nhận mail tại Settings.

68. Email có chống HTML injection không?

Có. Các field động được escape `&`, `<`, `>`, dấu nháy trước khi chèn vào HTML template.

69. Transporter được tạo mỗi email không?

Không. Service cache transporter trong runtime để tái sử dụng kết nối/cấu hình.

70. Email recipient hiện hỗ trợ nhiều user không?

Có ở mức tài khoản: backend lấy email profile của mọi user có `settings.emailEnabled=true`, loại trùng địa chỉ và dùng BCC khi có nhiều người nhận. Tuy nhiên hệ thống chưa ánh xạ device với owner, nên mọi user đang bật đều nhận anomaly của mọi thiết bị. `ALERT_RECIPIENT_EMAIL` chỉ fallback khi database chưa có user.

71. Throttle có tồn tại sau backend restart không?

Không, vì map nằm trong RAM. Muốn bền vững và dùng nhiều instance cần Redis hoặc collection riêng với atomic expiration.

72. Tại sao test dùng fake transport và fake clock?

Để test nhanh, lặp lại được và không gửi email thật. Fake clock cho phép nhảy chính xác 15 phút mà không phải chờ thực tế.

F. Câu hỏi phản biện và cách trả lời trung thực

73. Dự án đã hoàn toàn production-ready chưa?

Chưa. Core coursework và automated logic đã hoàn thành. Production cần persistent distributed state, per-user authorization/device ownership, observability, retry policy và end-to-end load/security testing.

74. Test pass có chứng minh relay dưới 500 ms không?

Không. Simulation chỉ chứng minh contract phần mềm. TC-06 phải đo từ thao tác web đến relay thật bằng log timestamp hoặc thiết bị đo.

75. Test email có chứng minh nhận mail dưới 5 giây không?

Không. Fake SMTP chứng minh logic severity và throttle. TC-05 với SMTP thật mới đo latency giao mail.

76. Tại sao không dùng machine learning phức tạp hơn?

Dataset nhỏ, thời gian dự án ngắn và yêu cầu cần giải thích được. Rolling Z-score minh bạch, nhẹ, dễ test và phù hợp ESP32/IoT coursework hơn một model khó hiệu chỉnh.

77. Tại sao không chạy Z-score trên ESP32?

Backend đã nhận và lưu lịch sử, có tài nguyên tốt hơn và dễ cập nhật threshold. ESP32 chỉ giữ logic an toàn bắt buộc phải hoạt động offline như gas threshold.

78. Nếu MongoDB lỗi thì email/anomaly có chạy không?

Trong pipeline hiện tại `SensorReading.create()` chạy trước anomaly engine, nên ghi telemetry lỗi sẽ dừng xử lý anomaly của message đó. Đây là tradeoff bảo đảm anomaly dựa trên dữ liệu đã persist. Edge gas vẫn an toàn cục bộ trên ESP32.

79. Nếu email đã gửi nhưng ghi anomaly log lỗi thì sao?

Action được thực hiện trước khi `AnomalyLog.create()` để lưu đầy đủ `actionsTaken`. Vì vậy có khả năng email đã gửi nhưng DB write sau đó lỗi. Thiết kế production tốt hơn có thể tạo log trạng thái PENDING trước rồi update action hoặc dùng outbox pattern.

80. Em sẽ mở file nào đầu tiên nếu được yêu cầu show code?

- CB-03: `backend/routes/control.js`, sau đó `mqttService.js`, `firmware/src/mqtt/mqtt_client.cpp`, `main.cpp` và `actuators/relay.cpp`.
- NC-01: `backend/services/anomalyEngine.js` tại `evaluateZScore` và `processTelemetryForAnomalies`.
- NC-03: `timeSeriesService.js`, `telemetryService.js`, rồi `routes/telemetry.js`.
- NC-05: `emailService.js` tại `sendAlertEmail`.

Câu hỏi bổ sung về MQ2 sau hiệu chỉnh

81. Tại sao `gasLevel` có thể là null?

Vì MQ2 chưa ở trạng thái `ready`. `null` biểu diễn đúng việc PPM chưa có giá trị, khác với `0` vốn là một phép đo. Backend vẫn lưu `sensorStatus.gas` nhưng không tạo gas anomaly hoặc email từ dữ liệu chưa hiệu chuẩn.

82. Vì sao cần burn-in 48 giờ và lưu `Ro` trong NVS?

Datasheet MQ2 yêu cầu preheat ban đầu trên 48 giờ. `Ro` là điện trở cảm biến trong không khí sạch; hiệu chuẩn ngay khi bật nguồn sẽ học một baseline sai. NVS giữ `Ro` qua reset, nên các lần sau chỉ warm-up 5 phút và không hiệu chuẩn sai lại từ đầu.

83. Trong 48 giờ burn-in có mất an toàn local không?

Không hoàn toàn. Firmware chưa được phép gọi điện áp raw là PPM hoặc gửi email, nhưng vẫn có ngưỡng ADC gần bão hòa để bật còi, LED và OLED tại chỗ. Đây là lớp dự phòng bảo thủ, không thay cho cảnh báo 600 PPM sau hiệu chuẩn.

84. Tại sao phải dùng FreeRTOS task riêng cho gas?

WiFi và bắt tay TLS có thể chặn Arduino `loop()` lâu hơn 200 ms. Task gas ưu tiên cao chạy mỗi 100 ms, chỉ đọc ADC và điều khiển local; task mạng publish bản ghi đã xếp hàng sau. Vì vậy đường an toàn không chờ cloud.

85. MQ2 có đo PPM chính xác cho mọi loại khí không?

Không. MQ2 phản ứng chéo với propane, methane, cồn và khói. Hệ thống nội suy đường propane trong datasheet nên phải gọi là LPG-equivalent PPM estimate. Muốn PPM chuẩn cần khí chuẩn, điều kiện nhiệt độ/độ ẩm kiểm soát và cảm biến chọn lọc phù hợp.

Code walkthrough phần Thái Kiệt

Tài liệu này không sao chép toàn bộ source. Nó trích các đoạn quyết định hành vi của CB-03, NC-01, NC-03 và NC-05, rồi giải thích cách các đoạn nối với nhau. Khi vấn đáp, nên mở file thật và tìm theo tên hàm in đậm.

1. Cách đọc project

Đọc một luồng theo thứ tự sau:

```
Input -> route/message handler -> service -> model/output -> test
```

Ví dụ CB-03:

```
Overview.jsx
  -> POST /api/v1/control/actuator
  -> control.js
  -> mqttService.publishControlCommand()
  -> MQTT topic
  -> MqttClient.handleMessage()
  -> main.cpp onControlMessage()
  -> Relay.on()/off()
  -> digitalWrite(GPIO 16 theo RELAY_ACTIVE_LOW)
```

2. CB-03 - Điều khiển quạt

2.1 Frontend gửi lệnh

File: `frontend/src/pages/Overview.jsx`

Đầu file có mapping giữa tên actuator của API và field trong telemetry:

```
const ACTUATOR_STATE_FIELDS = {
  fan: 'fanOn',
  buzzer: 'buzzerActive',
  alertLed: 'alertLedActive'
};
```

Mapping này thay cho ternary lồng nhau. Khi thêm actuator mới, chỉ cần thêm một entry và phần cập nhật optimistic state vẫn dễ đọc.

Hàm **handleActuatorToggle** thực hiện ba việc:

```
const targetState = !currentState;
const res = await apiFetch('/api/v1/control/actuator', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    deviceId: telemetry?.deviceId || 'safebox-01',
    actuator,
    state: targetState
  })
});
```

1. Đảo trạng thái hiện tại.
2. Gửi JSON đến backend.
3. Dùng `apiFetch` để tự gắn JWT.

Sau response thành công, UI cập nhật trước khi vòng polling tiếp theo chạy:

```
const stateField = ACTUATOR_STATE_FIELDS[actuator];
if (!stateField) throw new Error(`Unsupported actuator '${actuator}'`);
setTelemetry((prev) => ({
  ...(prev || {}),
  actuatorStates: {
    ...(prev?.actuatorStates || {}),
    [stateField]: targetState
  }
}));
```

Đây là optimistic update. Nó giúp UI phản hồi nhanh nhưng không được xem là xác nhận phần cứng.

2.2 JWT helper

File: `frontend/src/api.js`

```
export function apiFetch(input, options = {}) {
  const token = localStorage.getItem('safebox_token');
  const headers = new Headers(options.headers || {});

  if (token) headers.set('Authorization', `Bearer ${token}`);

  return fetch(input, {
    ...options,
    headers
  });
}
```

Điểm cần nói:

- Helper không tự tạo token; token được tạo bởi backend khi login.
- Nó giữ nguyên header caller truyền vào rồi bổ sung Authorization.
- Nếu không có token, backend middleware trả 401.

2.3 REST route nhận lệnh

File: `backend/routes/control.js`

Hàm **parseActuatorState** chuẩn hóa hai dạng input:

```
function parseActuatorState(value) {
  if (typeof value === 'boolean') return value;
  if (value === 'on') return true;
  if (value === 'off') return false;
  throw new TypeError("state must be a boolean or the string 'on'/'off'");
}
```

Route kiểm tra actuator trước khi publish:

```
if (!ALLOWED_ACTUATORS.has(actuator)) {
  return res.status(400).json({
    success: false,
    error: `Unsupported actuator '${actuator}'`
  });
}

const targetState = parseActuatorState(state);
const publishResult = await publishControlCommand(deviceId, actuator, targetState);
```

Nếu MQTT thật đang offline, service gắn code `MQTT_OFFLINE` và route trả 503. Input sai trả 400. Lỗi không dự kiến trả 500.

2.4 Kho trạng thái actuator

File: backend/services/actuatorStateService.js

```
const ACTUATOR_STATE_FIELDS = Object.freeze({
  fan: 'fanOn',
  buzzer: 'buzzerActive',
  alertLed: 'alertLedActive'
});
```

Hàm **updateActuatorState** lưu requested state sau publish:

```
function updateActuatorState(actuator, state) {
  const stateField = ACTUATOR_STATE_FIELDS[actuator];
  if (!stateField) throw new TypeError(`Unsupported actuator '${actuator}'`);

  currentActuatorStates = {
    ...currentActuatorStates,
    [stateField]: Boolean(state)
  };
  return getCurrentActuatorStates();
}
```

Hàm **synchronizeActuatorStates** thay snapshot bằng telemetry đã xác thực:

```
function synchronizeActuatorStates(actuatorStates) {
  currentActuatorStates = {
    fanOn: Boolean(actuatorStates.fanOn),
    buzzerActive: Boolean(actuatorStates.buzzerActive),
    alertLedActive: Boolean(actuatorStates.alertLedActive)
  };
  return getCurrentActuatorStates();
}
```

Service trả object copy, không trả reference map nội bộ. Caller không thể vô tình sửa state mà không đi qua service.

2.5 Backend publish MQTT

File: backend/services/mqttService.js

Hàm **publishControlCommand**:

```
const topic = `safebox/${deviceId}/control`;
const payload = JSON.stringify({ actuator, state: state ? 'on' : 'off' });
await new Promise((resolve, reject) => {
```

```
  mqttClient.publish(topic, payload, { qos: 1, retain: false }, (error) => {
    if (error) reject(error);
    else resolve();
  });
});
updateActuatorState(actuator, state);
return { status: 'published', topic };
```

Ý nghĩa từng option:

- `qos: 1`: at least once.
- `retain: false`: lệnh cũ không được giữ lại để chạy lại khi thiết bị kết nối sau.
- Chỉ cập nhật requested state sau callback publish không báo lỗi.

Khi telemetry thật quay về:

```
if (channel === 'telemetry') {
  const result = await ingestTelemetry(payload, deviceId);
  synchronizeActuatorStates(result.reading.actuatorStates);
  return result;
}
```

2.6 Firmware nhận MQTT

File: `firmware/src/mqtt/mqtt_client.cpp`

```
void MqttClient::handleMessage(byte* payload, unsigned int length) {
  JsonDocument doc;
  DeserializationError error = deserializeJson(doc, payload, length);
  if (error) return;

  const char* actuator = doc["actuator"] | "";
  bool state = false;
  if (doc["state"].is<bool>()) {
    state = doc["state"].as<bool>();
  } else {
    const char* stateStr = doc["state"] | "off";
    state = (strcmp(stateStr, "on") == 0);
  }

  if (_controlCallback) _controlCallback(actuator, state);
}
```

`MqttClient` chịu trách nhiệm subscribe topic control, parse JSON và đổi state sang boolean. JSON lỗi bị bỏ qua. Chuỗi khác `on` được xử lý theo hướng fail-off; actuator lạ không tạo tác động vì callback ở `main.cpp` không có nhánh tương ứng.

File: `firmware/src/main.cpp`

```
static void onControlMessage(const char* actuator, bool state) {
  if (strcmp(actuator, "fan") == 0) {
    if (state) fanRelay.on();
    else fanRelay.off();
    DBG("[Fan] Relay state: %s\n", fanRelay.isOn() ? "ON" : "OFF");
  } else if (strcmp(actuator, "buzzer") == 0) {
    edgeSafety.applyRemoteBuzzer(state);
  } else if (strcmp(actuator, "alertLed") == 0) {
    edgeSafety.applyRemoteAlertLed(state);
  }
}
```

`main.cpp` chỉ định tuyến actuator. Buzzer và alert LED đi qua `EdgeSafety`, nhờ đó cảnh báo gas cục bộ có thể ưu tiên hơn lệnh remote.

File: `firmware/src/actuators/relay.cpp`

```
void Relay::begin() {
  pinMode(_pin, OUTPUT);
  off();
}

void Relay::on() {
  digitalWrite(_pin, RELAY_ACTIVE_LOW ? LOW : HIGH);
  _state = true;
}

void Relay::off() {
  digitalWrite(_pin, RELAY_ACTIVE_LOW ? HIGH : LOW);
  _state = false;
}
```

Với module HL-52S mặc định `RELAY_ACTIVE_LOW=1`, GPIO 16 LOW là relay ON và HIGH là OFF. `begin()` gọi `off()`, nên fan vẫn tắt trong lúc boot.

3. NC-01 - Z-score và rule engine

File: `backend/services/anomalyEngine.js`

3.1 Hằng số và state

```
const WINDOW_SIZE = 30;
const DEDUPE_WINDOW_MS = Number(process.env.ANOMALY_DEDUPE_WINDOW_MS || 60_000);
const MIN_DELTA = { temperature: 1.0, humidity: 2.0 };
```

`DEFAULT_THRESHOLDS` được freeze để tránh sửa nhầm object mặc định. Runtime dùng một bản copy, nên `updateThresholds()` vẫn thay đổi được cấu hình đang chạy.

```
let systemThresholds = { ...DEFAULT_THRESHOLDS };
const rollingBuffers = new Map();
const anomalyDedupMap = new Map();
```

Key rolling buffer là `deviceId:sensorType`, vì mỗi thiết bị và mỗi metric cần baseline riêng.

3.2 Tính mean và standard deviation

```
function meanAndStdDev(values) {
  const mean = values.reduce((sum, value) => sum + value, 0) / values.length;
  const variance = values.reduce(
    (sum, value) => sum + ((value - mean) ** 2),
    0
  ) / values.length;
  return { mean, stdDev: Math.sqrt(variance) };
}
```

Code dùng population variance, chia cho N. Window 30 mẫu được coi là toàn bộ baseline hiện tại, không phải mẫu dùng để suy rộng thống kê học thuật.

3.3 Evaluate mẫu mới

```
if (!Object.hasOwn(MIN_DELTA, sensorType)) {
  return { ready: false, zScore: null, isAnomaly: false };
}
```

Dòng này là hàng rào bảo đảm gas và motion không bị đưa vào Z-score.

```
if (buffer.length === WINDOW_SIZE) {
  const { mean, stdDev } = meanAndStdDev(buffer);
  if (stdDev === 0) {
    result = { ready: true, zScore: 0, mean, stdDev, isAnomaly: false };
  } else {
    const zScore = (value - mean) / stdDev;
    const isAnomaly = Math.abs(zScore) > systemThresholds.zScoreLimit
      && Math.abs(value - mean) > MIN_DELTA[sensorType];
    result = { ready: true, zScore, mean, stdDev, isAnomaly };
  }
}

buffer.push(value);
if (buffer.length > WINDOW_SIZE) buffer.shift();
```

Thứ tự rất quan trọng: evaluate bằng 30 mẫu cũ, rồi mới roll mẫu mới vào window.

3.4 Bốn rule builder

Sau refactor, mỗi sensor có một hàm dễ mở và giải thích:

```
buildGasCandidate()
buildTemperatureCandidate()
buildMotionCandidate()
buildHumidityCandidate()
```

Hàm chính chỉ còn orchestration:

```
const context = { deviceId, timestamp, metrics };
const temperatureStats = evaluateZScore(deviceId, 'temperature', metrics.temperature);
const humidityStats = evaluateZScore(deviceId, 'humidity', metrics.humidity);
const candidates = [
  buildGasCandidate(context),
  buildTemperatureCandidate(context, temperatureStats),
  buildMotionCandidate(context),
  buildHumidityCandidate(context, humidityStats)
].filter(Boolean);

const created = await Promise.all(candidates.map(createAnomalyIfNew));
return created.filter(Boolean);
```

`candidate` là object trung gian mô tả anomaly và hành động cần làm. Chỉ sau dedupe và action handling nó mới thành MongoDB document.

3.5 Temperature rule

```
const aboveMaximum = metrics.temperature > systemThresholds.tempMax;
const belowMinimum = metrics.temperature < systemThresholds.tempMin;
```

Hard bounds được ưu tiên. Nếu trong bounds và automatic mode bật, code mới xét Z-score.

```
autoFan: metrics.temperature > statistics.mean
```

Z-score có thể âm hoặc dương. Fan chỉ bật khi bất thường theo hướng nóng lên, không bật khi nhiệt độ giảm mạnh.

3.6 Dedupe và action

```
function dedupeKey(candidate) {
  return `${candidate.deviceId}:${candidate.sensorType}:${candidate.triggerType}`;
}
```

```
const lastCreatedAt = anomalyDedupMap.get(key);
if (lastCreatedAt !== undefined && now - lastCreatedAt < DEDUPE_WINDOW_MS) {
  return null;
}
```

Key được reserve trước các thao tác async để hai message đến gần nhau không cùng vượt qua kiểm tra.

Action được bọc `try/catch` riêng. Nếu bật fan hoặc gửi email thất bại, anomaly vẫn được ghi và `actionsTaken` chứa lý do lỗi. Đây là graceful degradation.

4. NC-03 - Time Series ingestion

4.1 Schema

File: `backend/models/Schemas.js`

```
const SensorReadingSchema = new Schema({
  timestamp: { type: Date, required: true, default: Date.now },
  deviceId: { type: String, required: true, index: true },
  metrics: {
    temperature: { type: Number, required: true },
    humidity: { type: Number, required: true },
    gasLevel: { type: Number, required: true },
    lightIntensity: { type: Number, default: null },
    motionDetected: { type: Boolean, required: true }
  },
  actuatorStates: {
    fanOn: { type: Boolean, required: true, default: false },
    buzzerActive: { type: Boolean, required: true, default: false },
    alertLedActive: { type: Boolean, required: true, default: false }
  }
}, {
  autoCreate: false,
  autoIndex: false
});
```

`autoCreate=false` ngăn Mongoose tạo collection thường trước khi service có cơ hội tạo Time Series collection.

4.2 Tạo và verify collection

File: `backend/services/timeSeriesService.js`

```
const EXPIRE_AFTER_SECONDS = 30 * 24 * 60 * 60;
const TIME_SERIES_OPTIONS = {
  timeField: 'timestamp',
  metaField: 'deviceId',
  granularity: 'seconds'
};
```

Hàm **ensureSensorReadingsTimeSeries**:

1. Gọi `listCollections()` tìm `sensor_readings`.
2. Nếu chưa có, gọi `createCollection()` với `timeseries` và TTL.
3. Đọc metadata lại.
4. `validateCollectionOptions()` so từng option.
5. Sai cấu hình thì throw để startup fail rõ ràng.

4.3 Validate và ingest

File: `backend/services/telemetryService.js`

```
async function ingestTelemetry(payload, expectedDeviceId) {
  const normalized = validateTelemetryPayload(payload, expectedDeviceId);
  const reading = await SensorReading.create(normalized);
  const anomalies = await processTelemetryForAnomalies(reading);
  return { reading, anomalies };
}
```

`expectedDeviceId` lấy từ MQTT topic. Nếu payload khai một deviceId khác topic, validation từ chối để tránh ghi dữ liệu sai thiết bị.

Validation còn kiểm tra:

- Nhiệt độ -40 đến 85°C.
- Độ ẩm 0 đến 100%.
- Gas và light là số hữu hạn không âm.
- Motion và actuator states là boolean.
- Timestamp phải parse được.

4.4 Historical route

File: `backend/routes/telemetry.js`

```
let databaseQuery = SensorReading.find(query)
  .sort({ timestamp: -1 })
  .limit(parsedLimit);

const readings = await databaseQuery;
readings.reverse();
```

Hai bước này trả N mẫu mới nhất nhưng vẫn theo thứ tự thời gian phù hợp để vẽ chart.

5. NC-05 - Nodemailer và throttle

File: `backend/services/emailService.js`

5.1 Hằng số

```
const THROTTLE_WINDOW_MS = 15 * 60 * 1000;
const ALERT_SEVERITIES = new Set(['HIGH', 'CRITICAL']);
const emailThrottleMap = new Map();
```

5.2 Validate config

`getEmailConfig()` chỉ cho bật email khi đủ SMTP host, user và password. Port phải là integer từ 1 đến 65535. `SMTP_SECURE` mặc định true nếu port 465. `ALERT_RECIPIENT_EMAIL` không còn là recipient chính; nó chỉ fallback khi database chưa có user.

5.3 Lấy recipient từ user profile

File: `backend/services/userNotificationService.js`

```
const users = await User.find({})
  .select('email settings.emailEnabled')
  .lean();

const recipients = users
  .filter((user) => user.settings?.emailEnabled !== false)
  .map((user) => normalizeEmail(user.email));
```

Email nhận cảnh báo chính là `User.email` đã nhập khi đăng ký. Settings lưu `settings.emailEnabled`. Khi đã có user nhưng tất cả đều tắt, resolver trả danh sách rỗng và không dùng fallback, nhờ đó lựa chọn trên web có hiệu lực thật.

5.4 Gửi email

```
const severity = String(alert.severity || '').toUpperCase();
if (!ALERT_SEVERITIES.has(severity)) {
  return { status: 'ignored', reason: 'severity' };
}
```

MEDIUM và LOW dừng trước khi chạm SMTP.

```
const throttleKey = `${alert.deviceId}:${alert.sensorType}`;
const now = nowProvider();
```

```
const lastSent = emailThrottleMap.get(throttleKey);
if (lastSent !== undefined && now - lastSent < THROTTLE_WINDOW_MS) {
  return { status: 'throttled', lastSentAt: new Date(lastSent) };
}
```

Sau đó mới build message và gửi:

```
const providedRecipients = await recipientProvider(config);
if (providedRecipients.length === 0) {
  return { status: 'skipped', reason: 'no_recipients' };
}

const result = await activeTransporter.sendMail({
  from: config.from,
  ...recipientHeaders,
  ...message
});

emailThrottleMap.set(throttleKey, now);
```

Map chỉ được cập nhật sau khi `sendMail` resolve thành công.

Một recipient dùng header `to`; nhiều recipient dùng `bcc` để họ không nhìn thấy địa chỉ của nhau. Map throttle chỉ được cập nhật sau khi `sendMail` resolve thành công.

5.5 HTML safety

Mọi giá trị động được đưa qua `escapeHtml()` trước khi chèn vào HTML. Điều này ngăn dữ liệu cảm biến hoặc device ID chứa HTML tùy ý làm hỏng email.

Timestamp vẫn được lưu và xử lý nội bộ theo UTC. Riêng template email gọi `formatVietnamTimestamp()` với timezone `Asia/Ho_Chi_Minh`, nên người nhận thấy định dạng `YYYY-MM-DD HH:mm:ss UTC+7` mà không làm thay đổi dữ liệu gốc.

6. Test map

File test	Chứng minh
`actuatorStateService.test.js`	Mapping actuator và telemetry reconciliation
`anomalyEngine.test.js`	N=30, per-device, zero variance, min delta, hard rules, dedupe, integrated action
`emailService.test.js`	Severity filter, 15 phút, profile recipients, opt-out, HTML escaping
`userNotificationService.test.js`	Chuẩn hóa, loại trùng và lọc user đã tắt email
`userRoutes.test.js`	Settings lưu `settings.emailEnabled` đúng field
`telemetryService.test.js`	Payload validation, topic/device match, edge alert, actuator confirmation
`telemetryRoutes.test.js`	Latest-N query và chronological response
`timeSeriesService.test.js`	Collection options và fail-fast
`scripts/testCb03Mock.js`	REST fan ON/status/OFF có JWT trong simulation mode

6.1 MQ2 readiness contract cần biết khi giải thích telemetry

Firmware không còn biến mẫu ADC lúc MQ2 chưa ổn định thành PPM. Driver có năm trạng thái: `burn_in`, `warming_up`, `calibrating`, `ready`, `fault`. Chỉ `ready` mới gửi `metrics.gasLevel` dạng số; các trạng thái khác gửi `null` cùng `sensorStatus.gas` để backend vẫn lưu được chẩn đoán.

`telemetryService.js` kiểm tra tính nhất quán hai chiều: `ready=true` bắt buộc có PPM hữu hạn, còn chưa ready bắt buộc `gasLevel=null`. `anomalyEngine.js` bỏ qua gas khi giá trị không hữu hạn, nên dữ liệu burn-in không tạo anomaly/email giả. Firmware cũ và mock không có `sensorStatus` vẫn tương thích và được xem là legacy-ready.

Đường local safety độc lập với cloud: FreeRTOS task ưu tiên cao đọc MQ2 mỗi 100 ms. Sensor ready dùng ngưỡng 600 PPM; sensor đang chuẩn bị chỉ dùng ngưỡng điện áp ADC gần bão hòa để bật còi/LED/OLED tại chỗ. Raw fallback không publish MQTT vì nó không phải PPM đã hiệu chuẩn.

7. Điều nên và không nên học thuộc

Nên nhớ:

- Tên bốn luồng và thứ tự các tầng.
- Công thức Z-score, N=30, threshold 3.0 và minimum delta.
- Topic control và telemetry.
- QoS 1 là at least once.
- Time Series options và TTL 30 ngày.
- Email key và window 15 phút.

Không cần học thuộc:

- Toàn bộ HTML email template.
- Tất cả Tailwind class.
- Từng dòng setup MQTT/TLS.
- Giá trị ngẫu nhiên trong mock telemetry.

Khi không nhớ một chi tiết, mở đúng file và giải thích theo luồng dữ liệu. Khả năng định vị code và lý giải quyết định quan trọng hơn việc đọc thuộc source.

Kịch bản demo phần Thái Kiệt

Mục tiêu của kịch bản là tách rõ ba mức bằng chứng:

1. **Unit/integration simulation:** chạy được trên máy không cần ESP32, EMQX, Atlas hay SMTP thật.
2. **Build verification:** chứng minh frontend và firmware biên dịch được.
3. **Physical end-to-end:** chứng minh broker, database, email và relay thật hoạt động với latency yêu cầu.

Không dùng kết quả mức 1 để tuyên bố mức 3 đã pass.

1. Chuẩn bị trước demo

Phần mềm

- Node.js 18 trở lên.
- Dependencies đã cài bằng `npm ci` trong `backend` và `frontend`.
- PlatformIO để build/flash firmware.

Dịch vụ thật

- MongoDB Atlas connection string.
- EMQX host, username, password và CA certificate.
- SMTP account/app password và email nhận cảnh báo.

Phần cứng CB-03

- ESP32 DevKit V1.
- Relay HL-52S, signal nối GPIO 16.
- Fan DC dùng nguồn phù hợp, chung ground với relay/ESP32 theo sơ đồ nhóm.
- Không cấp nguồn fan trực tiếp từ GPIO ESP32.

2. Demo nhanh không cần phần cứng

Mở PowerShell:

```
cd C:\Users\ASUS\OneDrive\Documents\ChatGPT\Saferoom-IoT\backend
npm run test:kiet
```

Lệnh này chạy test tập trung cho CB-03, NC-01, NC-03, NC-05 và kết thúc bằng mô phỏng fan.
Kết quả bắt buộc:

```
# fail 0
CB-03 PASS: unauthorized access blocked; authenticated fan ON,
status, and fan OFF succeeded.
```

Nếu có test fail, không tiếp tục tuyên bố chức năng ổn. Đọc tên subtest và sửa nguyên nhân trước.

3. Show từng phần bằng test có mục tiêu

3.1 CB-03

```
npm run test:cb03
```

Luồng được chứng minh:

```
request không JWT -> 401
fan ON -> API success -> mock actuator fanOn=true
GET status -> fanOn=true
fan OFF -> API success
```

Điểm phải nói:

> *Đây là simulation test cho REST, auth, MQTT abstraction và state. Nó chưa đo relay thật.*

3.2 NC-01

```
node --test --test-name-pattern="Z-score|hard-rule|gas or motion|deduplicates"
↪  test/anomalyEngine.test.js
```

Các bằng chứng cần chỉ:

- 30 mẫu ổn định rồi spike.
- Window tách theo device và metric.
- Zero variance không chia cho 0.
- Minimum delta loại nhiễu nhỏ.
- Gas và motion không dùng Z-score.
- Gas/motion/humidity đi đúng hard-rule matrix.
- Dedupe loại anomaly lặp nhanh.

3.3 NC-03

```
node --test test/timeSeriesService.test.js test/telemetryService.test.js
↪  test/telemetryRoutes.test.js
```

Các bằng chứng cần chỉ:

- Tạo `sensor_readings` với Time Series options.

- TTL 30 ngày.
- Fail fast khi collection sai loại.
- Validate telemetry và topic/device mismatch.
- Historical query lấy N mẫu mới nhất và trả chronological order.

3.4 NC-05

```
node --test test/emailService.test.js
```

Các bằng chứng cần chỉ:

- HIGH gửi email.
- Cùng key bị throttle 15 phút.
- Device hoặc sensor khác có window riêng.
- MEDIUM không gọi SMTP.
- Dynamic HTML được escape.

3.5 Integrated NC-01 -> CB-03 -> NC-05

```
node --test --test-name-pattern="runs the Z-score spike" test/anomalyEngine.test.js
```

Test này tạo 30 mẫu nhiệt độ ổn định rồi đưa spike 35°C. Nó kiểm tra:

```
Z-score > 3
-> tạo Z_SCORE_EXCEEDED / HIGH
-> publish fan ON
-> tạo anomaly document
-> gửi email lần đầu
-> lần gửi cùng key tiếp theo bị throttled
```

4. Build verification

Frontend

```
cd C:\Users\ASUS\OneDrive\Documents\ChatGPT\Saferoom-IoT\frontend
npm run build
```

Kết quả bắt buộc là `built` và exit code 0. Cảnh báo bundle lớn không phải compile failure nhưng nên được ghi nhận.

Firmware

```
cd C:\Users\ASUS\OneDrive\Documents\ChatGPT\Saferoom-IoT\firmware
pio run
```

Kết quả bắt buộc là `[SUCCESS]`. Ghi lại phần trăm RAM và flash để trả lời câu hỏi tài nguyên.

5. Demo thật CB-03 / TC-06

5.1 Cấu hình

1. Copy `firmware/include/secrets.example.h` thành `firmware/include/secrets.h`.
2. Điền WiFi, EMQX host, port 8883, username, password và CA certificate.
3. Trong `backend/.env`, đặt:

```
MOCK_HARDWARE=false
DEVICE_ID=safebox-01
MQTT_BROKER_URL=mqtts://<host>:8883
MQTT_USERNAME=<username>
MQTT_PASSWORD=<password>
```

4. Bảo đảm backend và firmware dùng cùng `DEVICE_ID`.

5.2 Chạy

Terminal 1:

```
cd firmware
pio run --target upload
pio device monitor
```

Terminal 2:

```
cd backend
npm start
```

Terminal 3:

```
cd frontend
npm run dev
```

Mở `http://localhost:3000`, đăng nhập và vào Overview.

5.3 Bằng chứng cần quay/chụp

1. Dashboard hiển thị MQTT live và device online.
2. Nhấn fan ON.
3. Network tab cho thấy POST control trả 200 và `delivery=published`.
4. Serial Monitor in `[Fan] Relay state: ON`.
5. Relay click và fan quay.
6. Telemetry sau đó báo `fanOn=true`.
7. Lặp lại với OFF.

5.4 Đo latency

Mốc bắt đầu: thời điểm nhấn nút hoặc request rời browser.

Mốc kết thúc: thời điểm relay đổi trạng thái, có thể dùng video 60 fps hoặc timestamp đồng bộ giữa frontend và Serial Monitor.

Tiêu chí TC-06:

```
latency < 500 ms
```

Ghi ít nhất 5 lần ON/OFF, báo median và worst case. Không chỉ đo một lần đẹp nhất.

6. Demo thật NC-03 / TC-02

6.1 Kiểm tra ingestion

1. Chạy ESP32, EMQX và backend thật.
2. Mở MongoDB Atlas Data Explorer.
3. Theo dõi `sensor_readings` trong 20–30 giây.
4. Xác nhận timestamp cách nhau khoảng 2 giây và deviceId đúng.
5. Xác nhận metrics và actuatorStates có đủ field bắt buộc.

6.2 Kiểm tra collection options

Trong Atlas hoặc mongosh, kiểm tra metadata collection. Kết quả phải tương đương:

```
{
  "timeseries": {
    "timeField": "timestamp",
    "metaField": "deviceId",
    "granularity": "seconds"
  },
  "expireAfterSeconds": 2592000
}
```

6.3 Kiểm tra historical API

Sau khi đăng nhập, lấy JWT và gọi:

```
$token = '<JWT_FROM_LOGIN>'
$headers = @{ Authorization = "Bearer $token" }
Invoke-RestMethod `
  -Uri 'http://localhost:5000/api/v1/telemetry/historical?deviceId=safebox-01&limit=10' `
  -Headers $headers
```

Xác nhận response có tối đa 10 mẫu mới nhất, timestamp tăng dần.

7. Demo thật NC-01 / TC-04

Phương án an toàn

Không làm nóng DHT22 quá mức hoặc tạo điều kiện nguy hiểm. Dùng mock/test mode để trình bày thuật toán, hoặc dùng một script test telemetry kiểm soát trong môi trường demo riêng.

Trình tự cần chứng minh:

1. Bật automatic Z-score mode.
2. Nạp 30 mẫu ổn định quanh 25°C.
3. Nạp một mẫu spike đủ lớn nhưng vẫn hợp lệ, ví dụ 35°C.
4. Kiểm tra `anomaly_logs` có:

```
sensorType=temperature
triggerType=Z_SCORE_EXCEEDED
calculatedZScore > 3
severity=HIGH
```

5. Kiểm tra Overview hiển thị anomaly.
6. Kiểm tra action fan ON được publish nếu spike hướng lên.

Không chỉnh threshold gas local của ESP32 để giả lập Z-score; đây là hai cơ chế khác nhau.

8. Demo thật NC-05 / TC-05

8.1 Cấu hình SMTP

```
EMAIL_ENABLED=true
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=<smtp-user>
SMTP_PASS=<app-password>
SMTP_FROM=SafeRoom IoT Box <smtp-user>
# Chỉ dùng khi database chưa có user:
ALERT_RECIPIENT_EMAIL=<optional-fallback>
SMTP_VERIFY_ON_STARTUP=true
```

Không đưa `.env` hoặc app password lên màn hình quay demo.

8.2 Trình tự

1. Đăng nhập bằng tài khoản có email profile thật, mở Settings, bật `Nodemailer Email Alerts` và lưu.
2. Khởi động backend và xác nhận log `[Email] Alerts enabled.`.
3. Tạo một HIGH hoặc CRITICAL anomaly trong môi trường test an toàn.
4. Ghi timestamp anomaly.
5. Xác nhận email đến địa chỉ profile và nội dung đúng device, sensor, observed value, threshold.
6. Đo thời gian từ anomaly tới email; tiêu chí dưới 5 giây theo TC-05.
7. Tạo lại cùng `(deviceId, sensorType)` trong 15 phút.
8. Xác nhận không có email thứ hai và `actionsTaken` ghi `EMAIL_THROTTLED`.
9. Tạo anomaly khác sensor hoặc device và xác nhận vẫn gửi được.

9. Bảng ghi kết quả vật lý

Test	Lần	Bắt đầu	Kết thúc	Latency	Kết quả
TC-06 Fan ON	1				
TC-06 Fan OFF	1				
TC-06 Fan ON	2				
TC-06 Fan OFF	2				
TC-05 Email	1				
TC-05 Throttle	1			N/A	

10. Cách xử lý khi demo lỗi

API 401/403

- Đăng nhập lại.
- Kiểm tra localStorage có `safebox_token`.
- Không hardcode hoặc tắt auth để né lỗi.

API 503 MQTT offline

- Kiểm tra `MOCK_HARDWARE=false`.
- Kiểm tra broker URL phải là `mqtts://`.
- Kiểm tra username/password và CA.
- Kiểm tra log backend `connected/subscribed`.

Backend có MQTT nhưng device offline

- Kiểm tra WiFi ESP32.
- Kiểm tra device ID hai bên.
- Kiểm tra status topic và retained online payload.

Fan không quay nhưng Serial báo ON

- Kiểm tra relay wiring, common ground và nguồn fan.
- Đo GPIO 16.
- Kiểm tra relay active HIGH hay active LOW thực tế. Code hiện giả định HIGH=ON theo pinout dự án.

Không có email

- Kiểm tra `EMAIL_ENABLED` và đủ SMTP variables.
- Dùng app password, không dùng password tài khoản thường.
- Kiểm tra severity phải HIGH/CRITICAL.
- Kiểm tra có đang trong throttle window không.

Không có dữ liệu Atlas

- Kiểm tra MONGO_URI và network access list.
- Kiểm tra MQTT topic phải đúng `safebox/{id}/telemetry`.
- Kiểm tra backend log rejected payload do type/range/device mismatch.

Gas cao hoặc không có PPM

- Nếu web hiện `First burn-in`, giữ MQ2 cấp nguồn trong không khí sạch đủ 48 giờ; tiến độ được checkpoint vào NVS mỗi 10 phút.
- Nếu hiện `Warming up`, chờ 5 phút; firmware đang dùng lại `Ro` đã lưu.
- Nếu hiện `Calibrating`, không đưa cồn/khói/bật lửa gần cảm biến; firmware cần ít nhất 100 mẫu ổn định trong cửa sổ 60 giây.
- Nếu hiện `Sensor fault`, đo AO sau cầu chia áp và kiểm tra đúng 15 kΩ / 10 kΩ, common ground, VCC 5 V.
- TC-03 phải đo phản hồi local bằng phần cứng. Build pass không chứng minh thời gian thực tế dưới 200 ms.

11. Bằng chứng dự phòng

Trước ngày vấn đáp nên chuẩn bị:

- Video TC-06 có đồng thời màn hình web, Serial Monitor và relay/fan.
- Ảnh Atlas collection options và vài telemetry documents.
- Email thật có timestamp và nội dung alert.
- Terminal output `npm run test:kiet`.
- Terminal output firmware/frontend build.

Bằng chứng quay trước là phương án dự phòng khi WiFi hoặc cloud lỗi, không thay thế việc hiểu và giải thích code.